

Mise en page de base

Table des matières

Qu'est-ce qu'une interface utilisateur (UI) ?	2
Qu'est-ce que Jetpack Compose ?	2
Fonctions composables	2
Annotations	2
Noms des fonctions composables	3
Disposition des éléments de l'interface utilisateur	3
Syntaxe lambda de fin	4
Exemple complet	5
Taille de la police de caractères	8
Alignement du texte	8
Marge intérieure	8
Exercices	10
Références	10

Dans ce chapitre, **Jetpack Compose** est utilisé pour créer une application Android intégrant une interface utilisateur simple.

Qu'est-ce qu'une interface utilisateur (UI) ?

L'interface utilisateur d'une application correspond à ce que vous voyez à l'écran : texte, images, boutons et bien d'autres types d'éléments, ainsi que la façon de les organiser. En d'autres termes, il s'agit de la manière dont l'application présente les éléments à l'utilisateur et la façon d'interagir avec elle.

Une interface utilisateur est principalement composé :

- de boutons cliquables;
- d'étiquettes;
- de zone de texte;
- d'images.

Chacun de ces éléments est appelé un composant d'interface utilisateur. Presque tout ce que vous voyez à l'écran est un élément d'UI (également appelé composant d'UI). Ces éléments peuvent être interactifs, comme un bouton cliquable ou un champ de saisie modifiable, mais ils peuvent également être de simples images décoratives.

Qu'est-ce que Jetpack Compose ?

Jetpack Compose est un kit d'outils moderne permettant de créer des interfaces utilisateur Android. Compose simplifie et accélère le développement d'UI pour Android en utilisant moins de code, des outils puissants et des fonctionnalités Kotlin intuitives. Avec Compose, vous pouvez créer votre UI en définissant un ensemble de fonctions, appelées fonctions composables, qui reçoivent des données et émettent des éléments d'UI.

Fonctions composables

Les fonctions composables sont les éléments de base d'une UI dans Compose. Une fonction composable :

- décrit une partie de votre interface utilisateur ;
- ne renvoie aucune information ;
- reçoit des entrées et génère ce qui s'affiche à l'écran ;
- peut parfois émettre plusieurs éléments d'UI.

Annotations

Les annotations permettent d'ajouter des informations supplémentaires au code. Ces informations aident les autres développeurs et les outils tels que le compilateur Jetpack Compose à comprendre le code de l'application.

Une annotation est appliquée en ajoutant un préfixe, le caractère @, à son nom (l'annotation) au début de la déclaration. Différents éléments de code peuvent être annotés : les propriétés, les fonctions, les classes, etc.

Le code suivant présente un exemple de fonction annotée :

```
1 @Composable
2 fun Greeting(name: String) {
3     Text(text = "Hello $name!")
4 }
```

Noms des fonctions composables

Une fonction composable qui ne renvoie aucun élément et qui contient l'annotation `@Composable` DOIT être nommée en utilisant la casse Pascal. Il s'agit d'une convention d'attribution des noms selon laquelle chaque mot d'un mot composé commence par une majuscule. La différence la casse Pascal et le camel case est que dans le premier cas, tous les mots commencent par une majuscule. Dans le second, le premier mot commence par une minuscule.

La fonction Compose :

- *DOIT* être un nom : `DoneButton()`
- *NE DOIT PAS ÊTRE* un verbe ou un groupe verbal : `DrawTextField()`
- *NE DOIT PAS ÊTRE* une préposition accompagnée d'un nom : `TextFieldWithLink()`
- *NE DOIT PAS ÊTRE* un adjectif : `Bright()`
- *NE DOIT PAS ÊTRE* un adverbe : `Outside()`
- Les noms *PEUVENT* être précédés d'adjectifs descriptifs : `RoundIcon()`

Cette règle s'applique dans tous les cas, que la fonction émette ou non des éléments d'interface utilisateur.

Disposition des éléments de l'interface utilisateur

La hiérarchie de l'UI est basée sur des conteneurs. En d'autres termes, un composant peut contenir un ou plusieurs autres composants, et les termes "parent" et "enfant" sont parfois utilisés. Les éléments d'UI parents contiennent des éléments enfants, qui peuvent eux-mêmes contenir d'autres éléments enfants. Dans cette section, vous allez découvrir les composables `Column`, `Row` et `Box`, qui peuvent servir d'éléments d'interface utilisateur parents.

Dans Compose, les trois éléments de mise en page standard les plus simples sont les composables `Column`, `Row` et `Box`.

`Column`, `Row` et `Box` sont des fonctions composables qui peuvent recevoir des composables comme arguments, de sorte que vous pouvez placer des éléments dans ces conteneurs de mise en page.

Par exemple, les éléments enfants d'un composable **Row** sont placés horizontalement, les uns à la suite des autres, sur une même ligne.

```
1 Row {  
2     Text("First Column")  
3     Text("Second Column")  
4 }
```

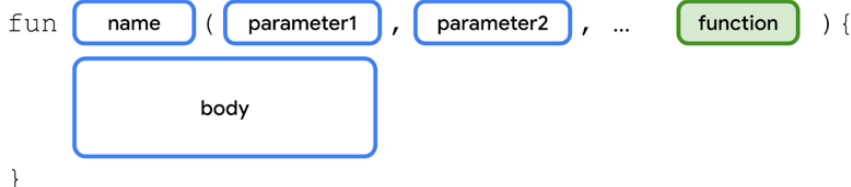
Tandis que **column** permet de placer les éléments verticalement.

La mise en page **Box** permet d'empiler des éléments les uns sur les autres. **Box** permet également de configurer l'alignement spécifique des éléments qu'elle contient.

Syntaxe lambda de fin

Dans l'extrait de code précédent, vous remarquerez que des accolades sont utilisées à la place des parenthèses dans la fonction composable **Row**. C'est ce qu'on appelle la *syntaxe "lambda de fin"*. Vous en apprendrez plus sur les *lambdas* et sur la syntaxe lambda de fin dans la suite de ce cours. Pour l'instant, familiarisez-vous avec cette syntaxe Compose communément utilisée.

Kotlin propose une syntaxe spécifique pour transmettre des fonctions sous forme de paramètres à d'autres fonctions, lorsque le dernier paramètre est justement une fonction.



```
fun name ( parameter1 , parameter2 , ... function ) {  
    body  
}
```

Si vous souhaitez transmettre une fonction en tant que paramètre, vous pouvez utiliser la syntaxe lambda de fin. Au lieu de placer le corps de la fonction avec le nom de la fonction entre parenthèses (**{}**), vous placez les parenthèses avec le corps de la fonction après son nom. Il s'agit d'une pratique communément utilisée dans Compose, il est donc recommandé de vous familiariser avec ce type de code.

Par exemple, le dernier paramètre de la fonction composable **Row()** est le paramètre **content**, qui émet les éléments d'UI enfants. Imaginons que vous souhaitiez créer une ligne avec trois éléments de texte. Le code suivant fonctionne, mais est très fastidieux à écrire :

```

1 Row(
2     content = {
3         Text("Some text")
4         Text("Some more text")
5         Text("Last text")
6     }
7 )

```

Étant donné que le paramètre `content` est le dernier du prototype de la fonction et que vous transmettez sa valeur sous la forme d'une expression lambda (pour l'instant, ce n'est pas grave si vous ne savez pas ce qu'est un lambda, familiarisez-vous simplement avec la syntaxe), vous pouvez supprimer le paramètre `content` et les parenthèses comme suit :

```

1 Row {
2     Text("Some text")
3     Text("Some more text")
4     Text("Last text")
5 }

```

Exemple complet

Pour cet exemple, une application permettant de faire la conversion de pouces à centimètres sera développée. Pour ce faire, il est nécessaire de configurer un projet dans Android Studio avec un modèle Activité Compose vide. Nommez ce projet `Convertisseur`

1. Dans le fichier `MainActivity.kt`, supprimez la définition de la fonction `Greeting()`. Dans la suite de l'atelier de programmation, vous ajouterez votre propre fonction pour afficher le message d'accueil.
2. Ajoutez une fonction appelée `Convertisseur()`. N'oubliez pas d'ajouter l'annotation `@Composable` avant la fonction, car il s'agit d'une fonction `Compose`.

```

1 package com.example.convertisseur
2
3 import android.os.Bundle
4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.compose.foundation.layout.Arrangement
7 import androidx.compose.foundation.layout.Column
8 import androidx.compose.foundation.layout.Row
9 import androidx.compose.foundation.layout.Spacer
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.height
12 import androidx.compose.foundation.layout.padding
13 import androidx.compose.material3.Button
14 import androidx.compose.material3.ExperimentalMaterial3Api
15 import androidx.compose.material3.MaterialTheme
16 import androidx.compose.material3.OutlinedTextField

```

```

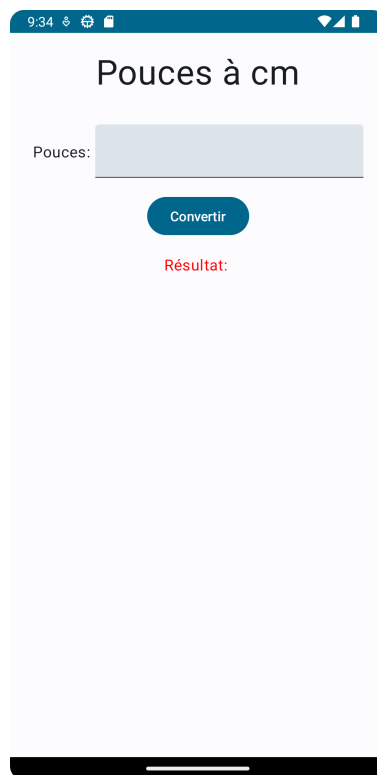
17 import androidx.compose.material3.Surface
18 import androidx.compose.material3.Text
19 import androidx.compose.material3.TextField
20 import androidx.compose.runtime.Composable
21 import androidx.compose.ui.Alignment
22 import androidx.compose.ui.Modifier
23 import androidx.compose.ui.graphics.Color
24 import androidx.compose.ui.tooling.preview.Preview
25 import androidx.compose.ui.unit.dp
26 import androidx.compose.ui.unit.sp
27 import com.example.convertisseur.ui.theme.ConvertisseurTheme
28
29 class MainActivity : ComponentActivity() {
30     @OptIn(ExperimentalMaterial3Api::class)
31     override fun onCreate(savedInstanceState: Bundle?) {
32         super.onCreate(savedInstanceState)
33         setContent {
34             ConvertisseurTheme {
35                 Surface(
36                     modifier = Modifier.fillMaxSize(),
37                     color = MaterialTheme.colorScheme.background
38                 ) {
39                     Convertisseur("Pouces à cm")
40                 }
41             }
42         }
43     }
44 }
45
46 @ExperimentalMaterial3Api
47 @Composable
48 fun Convertisseur(name: String, modifier: Modifier = Modifier) {
49     Column(
50         modifier = modifier.fillMaxSize(),
51         verticalArrangement = Arrangement.Top,
52         horizontalAlignment = Alignment.CenterHorizontally
53     ){
54         Text(
55             text = "$name",
56             fontSize = 36.sp,
57             modifier = Modifier.padding(
58                 start = 16.dp,
59                 top = 16.dp,
60                 end = 16.dp,
61                 bottom = 32.dp
62             )
63         )
64     }
65     Row(
66         verticalAlignment = Alignment.CenterVertically,
67     ){
68         Text(
69             text = "Pouces: ",
70

```

```

71         TextField(
72             value = "",
73             onChange = { /*TODO*/ }
74         )
75     }
76     Spacer(Modifier.height(16.dp))
77     Button(onClick = { /*TODO*/ }) {
78         Text(text = "Convertir")
79     }
80     Spacer(Modifier.height(16.dp))
81     Text(
82         text = "Résultat: ",
83         color = Color.Red,
84     )
85 }
86 }
87
88 @OptIn(ExperimentalMaterial3Api::class)
89 @Preview(showBackground = true)
90 @Composable
91 fun ConvertisseurPreview() {
92     ConvertisseurTheme {
93         Convertisseur("Pouces à cm")
94     }
95 }

```



L'application est composée de 3 étiquettes (Ligne 55, 68 et 81), d'une zone de texte 71, d'un bouton (ligne 77).

ATTENTION!

L'ajout de la zone de texte pourrait générer une erreur. Si le message d'erreur suivant apparaît: *This material API is experimental and is likely to change or to be removed in the future.*, il est nécessaire d'ajouter les instructions suivantes dans le fichier **build.gradle**

```
1 kotlinOptions {  
2     allWarningsAsErrors = false  
3     freeCompilerArgs += [  
4         '-opt-  
        in=androidx.compose.material3.ExperimentalMaterial3Api'  
5     ]  
6 }
```

Le message d'erreur devrait disparaître après avoir synchronisé le *Gradle* ou après avoir redémarrer Android Studio.

Source: <https://stackoverflow.com/questions/71069737/this-material-api-is-experimental-and-is-likely-to-change-or-to-be-removed-in-th>

Les 2 espaceurs (Ligne 76 et 80) sont utilisé pour espacer les éléments entre eux. Il est possible d'utiliser des marges pour arriver au même résultat.

Taille de la police de caractères

Il est possible de modifier la taille de la police de caractères d'un étiquette en assignant la valeur **36.sp** au paramètre **fontSize** (ligne 57).

Les *pixels indépendants de l'échelle* (SP) sont une unité de mesure de la taille de la police. Dans les applications Android, les éléments textuels de l'UI peuvent utiliser deux unités de mesure différentes : les *pixels indépendants de la densité* (DP), que vous utiliserez plus tard pour la mise en page, et les pixels indépendants de l'échelle (SP). Par défaut, l'unité en SP est identique à celle en DP, mais elle s'adapte aux préférences de taille de police définies par l'utilisateur dans les paramètres de son téléphone.

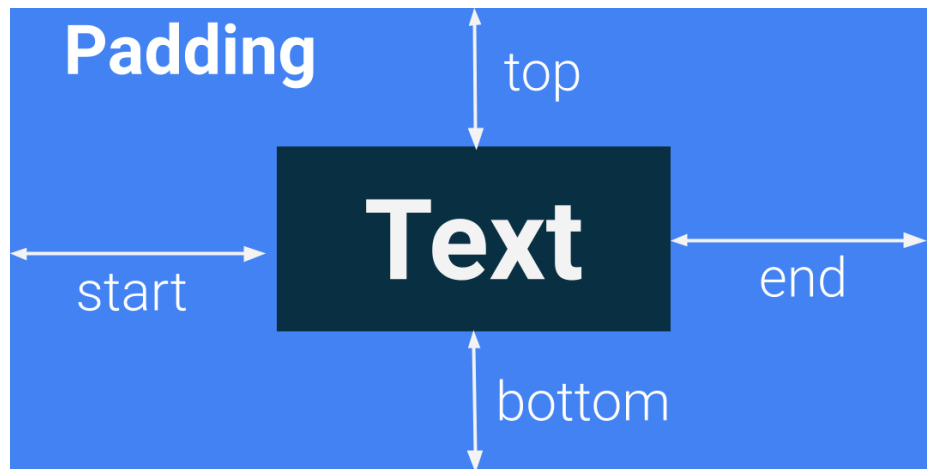
Alignement du texte

À la ligne 50, la valeur **modifier.fillMaxSize()** est attribué au paramètre **modifier**. La hauteur et la largeur de la disposition en colonnes seront définies sur la hauteur et la largeur maximales disponibles.

Les propriétés **verticalArrangement** et **horizontalAlignment** au ligne 51 et 52 permettent de placer les éléments de texte en haut de l'alignement pour les centrer horizontalement.

Marge intérieure

Un élément d'interface utilisateur s'encapsule autour de son contenu. Pour éviter qu'il ne s'encapsule trop, vous pouvez spécifier une marge intérieure *de chaque côté*.



La marge intérieure est utilisée comme modificateur, ce qui signifie que vous pouvez l'appliquer à n'importe quel composable. Pour chaque côté du composable, le modificateur padding utilise un argument facultatif qui définit la quantité de marge intérieure.

Dans la fonction modulable du message d'anniversaire, ajoutez un argument `modifier` et définissez-le sur `Modifier.padding()`, puis transmettez-lui un argument nommé `top` défini sur `16.dp`.

Exercices

Exercice 1

Refaire l'exemple des notes de cours

Exercice 2

Créer l'interface suivante.



Références

- Créer une mise en page de base: <https://developer.android.com/courses/pathways/android-basics-compose-unit-1-pathway-3?hl=fr>
- Jetpack Compose Tutorial: <https://www.jetpackcompose.net/>